

MusicXD — Documentação de Arquitetura de Software

Resumo executivo

Este relatório propõe uma arquitetura “10/10” para o **MusicXD**, uma plataforma social de descoberta musical inspirada no Letterboxd (para música), incorporando sua ideia original de **API central + armazenamento desnormalizado** e consolidando recomendações de **modular monolith**, **Clean Architecture**, **DDD por módulo**, **sincronização assíncrona com Spotify** e um **pipeline de eventos** para suportar feed social e leaderboards com baixa latência.

As decisões centrais são:

1) **Modular Monolith (MVP e até ~100k MAU)**: um único deploy, porém com módulos bem definidos (Users, Auth, MusicCatalog, SpotifySync, Reviews, Feed, Analytics, Chat) e limites “DDD-like” (bounded contexts). Isso reduz complexidade operacional e mantém alta qualidade de engenharia (testabilidade, separação de responsabilidades), alinhado a práticas de DDD/Bounded Context e Clean Architecture/Ports & Adapters. ¹

2) **Spotify como fonte externa + espelhamento local do catálogo**: persistir **Artist/Album/Track** com `spotify_id` e metadados essenciais, reduzindo latência e dependência de chamadas externas, ao mesmo tempo respeitando políticas de atribuição, linking e uso de conteúdo. ²

3) **Sincronização via background jobs**: usar Authorization Code Flow (com PKCE quando aplicável) e **refresh_token** para buscar “recently played” e “top items” de forma assíncrona, respeitando rate limits (429 + Retry-After) e variações de quota. ³

4) **Leaderboards e Analytics desnormalizados com eventos**: publicar eventos de domínio (ex.: `TrackReviewed`, `AlbumReviewed`, `SpotifyHistorySynced`) em mensageria (Kafka/RabbitMQ) com padrão **Transactional Outbox** para consistência; workers atualizam tabelas agregadas (ex.: top do mês, ranking global e ranking de amigos) em stores apropriadas (Redis / ClickHouse / Elasticsearch conforme o caso). ⁴

5) **Feed social com estratégia híbrida fan-out/fan-in**: começar com fan-out on write para “usuários normais” e fallback/híbrido para accounts de alto fanout; isso segue padrões conhecidos em sistemas de timeline que usam serviço de fanout + cache corporativo (ex.: práticas descritas publicamente por engenharia do X). ⁵

6) **Chat em tempo real com WebSocket/SignalR + Redis**: WebSocket é o protocolo padrão para comunicação bidirecional; SignalR escala horizontalmente usando Redis backplane; Socket.IO é alternativa com fallback para long-polling quando WebSocket não estiver disponível. ⁶

Para 100k MAU, a recomendação prática é: **PostgreSQL como fonte de verdade**, **Redis como cache/leaderboard hot path**, e (opcional) **ClickHouse** quando análises e ranking por janelas temporais/segmentos começarem a exigir OLAP real (consultas agregadas com alta cardinalidade). ⁷

Visão do produto e requisitos

Visão do produto

O MusicXD deve funcionar como uma rede social de música orientada a **opiniões e descoberta**: usuários avaliam **músicas e álbuns**, seguem amigos, veem um feed com atividade, exploram rankings e descobrem faixas a partir de afinidade social. O diferencial do produto é “descoberta por comunidade” e não streaming.

A integração com Spotify adiciona: (a) identidade/conexão de conta, (b) insumos de escuta (recently played, top tracks/artists) para enriquecer recomendações e “cartões sociais” de atividade, via OAuth 2.0 e escopos específicos. ⁸

Requisitos funcionais

Abaixo, requisitos funcionais mínimos (MVP) e evolução natural. Onde aplicável, há dependência explícita de endpoints e permissões da API do Spotify (ex.: recently played, top items). ⁹

Conta e autenticação - Cadastro/login (e-mail/senha ou social login) + perfil público. - Conectar conta Spotify (OAuth), armazenando refresh_token com segurança para sincronizações futuras. ¹⁰

Reviews e notas - Avaliar música (nota + texto) e avaliar álbum (nota + texto). - Curti/Comentários em reviews. - Página de detalhes: música/álbum com reviews agregadas + reviews de amigos em destaque.

Social - Seguir amigos (follow) e ver atividade deles. - Ranking “entre amigos” (músicas/álbuns mais bem avaliados no círculo).

Feed - Feed de eventos: review criada, review curtida, novo follow, batidas de “top tracks do mês” do usuário (opcional), etc. - Filtros do feed (ex.: só amigos, só reviews, etc.).

Leaderboards - Leaderboard global do mês: top 10 músicas do mês, top 10 álbuns do mês. - Leaderboard de amigos. - Rankings por período (ex.: últimos 30 dias / mês calendário).

Chat - Chat 1:1 entre usuários (mínimo), com entrega em tempo real e histórico.

Requisitos não-funcionais

Desempenho e experiência - Baixa latência em “páginas quentes”: feed e leaderboards devem responder rapidamente com cache/agregação (motivo para desnormalização). ¹¹
- Resiliência a rate limit do Spotify (429 + Retry-After) e a variações de quota. ¹²

Confiabilidade (SLO) - SLOs devem ser explicitados como objetivos medidos por SLIs (ex.: disponibilidade / latência p95), com uso de error budgets para guiar trade-offs de entrega vs estabilidade. ¹³

Segurança e privacidade - OAuth 2.0 corretamente implementado; quando cliente não pode guardar client_secret, usar PKCE no code flow. ¹⁴
- JWT como mecanismo de sessão/claims (com validação robusta) e rate limiting para conter abuso/ consumo irrestrito. ¹⁵

Compliance com políticas do Spotify - Se exibir metadados/arte do Spotify, cumprir atribuição e link back, e respeitar restrições (ex.: não oferecer conteúdo como “serviço de metadados” isolado). ¹⁶

Arquitetura de referência

Decisão principal: Modular Monolith por bounded contexts

Para 100k MAU, o **ponto ótimo** custo/complexidade é um **modular monolith**: um único binário/deploy, mas com módulos e contratos internos rígidos (interfaces, eventos e regras de dependência). Isso permite evoluir para microservices no futuro sem “big rewrite”, ao mesmo tempo em que evita custo operacional precoce (observabilidade, rede, deployments independentes, governança de contratos). Essa decomposição por contexto se alinha diretamente ao conceito de **Bounded Context** em DDD. ¹⁷

Módulos do MusicXD e responsabilidades

- **Users**: perfil, follow graph, preferências, privacidade.
- **Auth**: login, sessão, JWT, refresh do próprio MusicXD.
- **MusicCatalog**: catálogo espelhado (Artist/Album/Track), lookup, busca interna.
- **SpotifySync**: OAuth Spotify, tokens, jobs de sync, normalização de dados vindos do Spotify.
- **Reviews**: avaliações e comentários; cálculo do score canônico (fonte de verdade).
- **Feed**: geração/consulta do feed (com estratégia híbrida fan-out/fan-in).
- **Analytics**: agregados (top do mês, trending, leaderboards global/friends) e materialized views.
- **Chat**: mensagens, conversas, entrega real-time.

Essa separação é a tradução prática de DDD estratégicos: cada módulo é um “microserviço potencial”, mas inicialmente no mesmo deploy. ¹⁸

Diagrama de alto nível

```
flowchart TB
    UI[Frontend Web/Mobile] --> API[MusicXD API (Modular Monolith)]

    subgraph API [MusicXD API]
        Users[Users]
        Auth[Auth]
        Catalog[MusicCatalog]
        Sync[SpotifySync]
        Reviews[Reviews]
        Feed[Feed]
        Analytics[Analytics]
        Chat[Chat]
    end

    end

    API --> PG[(PostgreSQL - fonte de verdade)]
    API --> Redis[(Redis - cache/leaderboards/presença)]
    API --> ES[(Elasticsearch - busca/descoberta opcional)]
    API --> CH[(ClickHouse - analytics pesado opcional)]
    API --> MQ[(Mensageria: Kafka ou RabbitMQ)]
```

```
Sync --> Spotify[Spotify Web API]
MQ --> Workers[Workers (sync/agregação/feed)]
Workers --> PG
Workers --> Redis
Workers --> CH
```

PostgreSQL é recomendado como fonte de verdade para dados transacionais; ClickHouse (colunar/OLAP) entra quando consultas analíticas frequentes exigirem performance e custo previsível para agregações em alta escala. ¹⁹

Clean Architecture e Ports & Adapters dentro de cada módulo

A **regra de dependência** é a mesma para todos os módulos: domínio não depende de infraestrutura, e integrações externas entram como adapters. Isso coincide com Clean Architecture e com Ports & Adapters/Arquitetura Hexagonal: “ports” (interfaces) no core; “adapters” (DB, APIs externas, mensageria) na borda. ²⁰

Estrutura típica por módulo (exemplo conceitual): - **Domain**: entidades, agregados, value objects, invariantes (regras). - **Application**: casos de uso (commands/queries), DTOs, validações, orquestração. - **Infrastructure**: repositórios (Postgres), clients (Spotify), publishers (Kafka/RabbitMQ), cache (Redis). - **Interfaces**: controllers REST/GraphQL, hubs de chat, serialização.

Essa organização é reforçada por documentação e análise de domínio (DDD) e bounded contexts (cada contexto com modelo próprio e integração explícita). ²¹

Dados e integração com Spotify

Modelo de dados normalizado (fonte de verdade)

O modelo normalizado serve para: - consistência transacional (reviews, follows, autorização), - integridade referencial (track/album/artist -> reviews), - auditoria (eventos/outbox).

PostgreSQL é um DB relacional amplamente usado em workloads complexos e transacionais (base para consistência e integridade). ²²

Tabelas principais (exemplo base) - `users`: identidade pública e configurações. - `user_follow`: follow graph. - `reviews_track`: reviews/nota para música. - `reviews_album`: reviews/nota para álbum. - `review_likes`, `review_comments`. - `chat_threads`, `chat_messages`. - `spotify_accounts`: vínculo com Spotify + tokens. - `outbox_events`: fila transacional de eventos a publicar.

Espelhamento do catálogo Spotify (Artist, Album, Track com spotify_id)

A recomendação “10/10” é: MusicCatalog mantém **espelhamento local mínimo** do catálogo necessário (lookup e exibição) e usa Spotify como fonte de enriquecimento quando preciso. Isso se apoia na existência de IDs estáveis e endpoints de consulta do catálogo (ex.: `GET /tracks/{id}`) e no fato de que o app tipicamente precisa renderizar muitas páginas sem depender de latência externa. ²³

Esquema sugerido (normalizado) - `artist` - `id` (UUID interno) - `spotify_id` (unique) - `name` - `image_url` (opcional; respeitar guidelines) - `created_at`, `updated_at` - `album` - `id` - `spotify_id` (unique) - `artist_id` (FK -> artist) - `name` - `release_date` (string/data conforme retorno) - `image_url` - `created_at`, `updated_at` - `track` - `id` - `spotify_id` (unique) - `album_id` (FK -> album) - `name` - `duration_ms` - `track_number` - `explicit` (bool) - `created_at`, `updated_at` - `track_artist` (N:N quando houver feats/múltiplos artistas) - `track_id`, `artist_id`, `role`

Observação de compliance: se o MusicXD exibir metadados/capa do Spotify, deve atribuir corretamente e linkar de volta ao item correspondente no Spotify; além disso, não deve “revender” metadados/capas como produto standalone. ²⁴

OAuth, escopos e estratégia de tokens

Spotify implementa OAuth 2.0; para app com sessão longa, o Authorization Code Flow é recomendado; e quando não se pode armazenar `client_secret` em segurança (ex.: mobile), deve-se usar PKCE. ²⁵

Access tokens expiram (documentação do Spotify descreve expiração e o uso de refresh tokens); `refresh_token` permite obter novos access tokens sem reautenticação do usuário. ²⁶

Escopos típicos para MusicXD - `user-read-recently-played` para histórico recente (base do “últimas ouvidas”). ²⁷
- `user-top-read` para top tracks/artists (base do “música mais escutada / top do mês”). ²⁸

Rate limits e resiliência Spotify documenta rate limit em janela móvel e sinaliza 429 com Retry-After; o limite varia por modo (development vs extended quota), então o desenho de sync deve ser tolerante (retry/backoff/jitter e fila). ¹²

Sincronização: on-demand vs background jobs (comparativo)

A documentação do Spotify reforça flows e conceitos de tokens/refresh; somado ao rate limit, o padrão mais robusto para produto social é **background sync** com jobs idempotentes e controle de backlog. ²⁹

Estratégia	Como funciona	Vantagens	Riscos/Desvantagens	Quando usar
On-demand	Buscar no Spotify quando o usuário abre telas	Implementação simples; menor storage	Latência variável; custo de chamadas; rate limit em picos	MVP muito simples, pouca carga
Background jobs	Jobs periódicos (ex.: a cada X horas) + “sync on connect”	UX consistente; amortiza chamadas; controla rate limit	Exige scheduler/worker; estado de jobs	Recomendado para 100k MAU

O “10/10” sugerido: **background como default**, com on-demand apenas para “cache miss” ou enriquecimento pontual. ³⁰

Diagrama do fluxo Spotify (autorização + sync)

```
sequenceDiagram
    participant U as Usuario
    participant FE as Frontend
    participant API as MusicXD API (SpotifySync)
    participant SP as Spotify OAuth/API
    participant W as Worker Sync
    participant DB as PostgreSQL

    U->>FE: Conectar Spotify
    FE->>API: GET /spotify/connect (gera URL, state, code_challenge)
    API->>SP: Redirect para authorize (Authorization Code Flow + PKCE)
    SP->>FE: Redirect com code
    FE->>API: POST /spotify/callback {code, state, code_verifier}
    API->>SP: Troca code por tokens
    SP->>API: access_token + refresh_token
    API->>DB: Salva refresh_token (criptografado) + conta vinculada
    API->>DB: Enfileira job "SYNC_SPOTIFY_INITIAL"

    W->>DB: Pega job
    W->>SP: Chama endpoints (recently played / top items)
    SP->>W: Dados
    W->>DB: Persiste listening snapshots + atualiza MusicCatalog (upsert Artist/Album/Track)
```

Feed, rankings, analytics e pipeline de eventos

Modelo de feed: fan-out vs fan-in (e decisão recomendada)

Existem dois padrões clássicos para feed social:

- **Fan-out on write:** ao produzir um evento (ex.: review), você pré-computa e grava esse item nos feeds dos seguidores. Reads ficam rápidos, mas writes amplificam com muitos seguidores.
- **Fan-in on read:** ao ler o feed, você consulta eventos dos produtores seguidos e monta na hora. Writes são baratos, mas reads podem ficar caros (joins/merge, ranking, paginação).

Uma prática comum é o **híbrido** (fan-out para maioria, estratégia especial para “alto fanout”). Sistemas de timeline de grande escala tornaram público o uso de serviços específicos de fanout e caches para timelines; a engenharia do X descreveu componentes como Fanout Service e caches baseados em Redis para timelines. ⁵

Recomendação 10/10 para 100k MAU - Implementar **fan-out on write** como default para usuários comuns (simplicidade e performance). - Aplicar **limiar de fanout** (ex.: acima de N seguidores) para mudar para híbrido (armazenar evento em “author events” e mesclar no read). - Cachear páginas do feed (cursor-based) em Redis para reduzir carga em picos.

Tabela Feed (normalizado para auditoria + suporte ao híbrido)

Criar uma tabela **canônica** de eventos e (opcionalmente) uma tabela “materializada” de feed por usuário.

`feed_event` (**canônico**) - `id` - `actor_user_id` - `verb` (enum: REVIEW_TRACK_CREATED, REVIEW_ALBUM_CREATED, FOLLOW_CREATED, REVIEW_LIKED...) - `object_type` (TRACK/ALBUM/REVIEW/USER) - `object_id` - `created_at` - `metadata` (jsonb: rating, snippet, etc.)

`feed_inbox_item` (**materializado, fan-out on write**) - `inbox_user_id` - `feed_event_id` - `created_at` - índice (`inbox_user_id, created_at desc`) para paginação

A separação canônico vs inbox permite: - auditoria e reprocessamento (se regras mudarem), - feed rápido (inbox), - estratégia híbrida (quando não fan-out, o read mescla `feed_event` do autor). ³¹

Rank/Leaderboards: design desnormalizado orientado a eventos

Aqui entra sua ideia original (muito correta) de uma “estrutura desnormalizada” para top 10 do mês, mais escutadas, etc. A recomendação “10/10” é formalizar isso com:

- 1) Eventos de domínio (ex.: `TrackReviewed`)
- 2) Um pipeline confiável (Outbox + broker)
- 3) Workers idempotentes que mantêm **tabelas agregadas**

Esse desenho é compatível com Event Sourcing como conceito (eventos como sequência de mudanças) e também com CQRS (modelo de escrita e leitura separados), desde que aplicado com cuidado para não adicionar complexidade desnecessária. ³²

Tabelas agregadas (exemplos) - `agg_track_rating_month` - `month` (YYYY-MM) - `track_id` - `rating_sum` - `rating_count` - `avg_rating` (derivado) - `updated_at` - `agg_album_rating_month` (similar) - `leaderboard_global_month` - `month` - `entity_type` (TRACK/ALBUM) - `entity_id` - `rank` - `score` - `leaderboard_friends_month` - `month` - `user_id` - `entity_type` - `entity_id` - `rank` - `score`

Redis para leaderboards: usar Sorted Sets para top-N e paginação rápida (hot path). **ClickHouse** entra quando você quer análises mais profundas (segmentação por país, coortes, janelas móveis, trending com múltiplas features) com alto volume de eventos e consultas agregadas complexas. ³³

Armazenamento desnormalizado: Redis vs ClickHouse vs Elasticsearch (comparativo)

Opção	Melhor para	Ponto forte	Limitação típica	Recomendação para MusicXD
Redis	Cache, top-N, presença, rate limiting, sorted sets	Latência muito baixa; estruturas práticas; Pub/Sub	Pub/Sub tem semântica at-most-once e não é fila durável (não usar como “broker definitivo”) ³⁴	Use para leaderboards quentes + cache do feed

Opção	Melhor para	Ponto forte	Limitação típica	Recomendação para MusicXD
ClickHouse	OLAP/ analytics em alta escala	Colunar e otimizado para agregações e relatórios em “tempo quase real”	Não é o melhor sistema para OLTP/ transações lineares ³⁵	Use quando analytics crescer (fase 2/3)
Elasticsearch	Busca textual, discovery, ranking de texto	Full-text search e pipeline de análise de texto	Não substitui OLTP; tuning/infra pode ser custoso ³⁶	Use para busca de catálogo/reviews e discovery textual

Pipeline de eventos e mensageria

Para suportar feed e leaderboards com confiabilidade, recomenda-se:

- Publicar eventos após commit de escrita.
- Evitar “dual writes” (DB + broker) sem coordenação: usar **Transactional Outbox** (tabela outbox no mesmo banco da operação) e um publisher assíncrono. Isso é um padrão amplamente recomendado para consistência em arquiteturas distribuídas. ³⁷

Diagrama do pipeline (Outbox + broker + consumers)

```

flowchart LR
    A[Use case: CreateTrackReview] --> DB[(PostgreSQL)]
    DB --> O[Outbox table]
    O --> P[Outbox Publisher]
    P --> MQ[Kafka ou RabbitMQ]
    MQ --> C1[Consumer: FeedWorker]
    MQ --> C2[Consumer: RankingAggregator]
    MQ --> C3[Consumer: NotificationWorker]
    C1 --> Redis[(Redis Feed Cache)]
    C2 --> Agg[(Aggregated Tables / ClickHouse)]
    C3 --> RT[Realtime Gateway (Chat/Notifications)]

```

Kafka vs RabbitMQ: comparação prática para o MusicXD

Kafka é descrito como plataforma de **event streaming** para pipelines e processamento de eventos; RabbitMQ é um broker de mensageria com modelo de filas/roteamento (AMQP). A escolha depende do que mais pesa: replay e retenção (Kafka) vs roteamento e simplicidade (RabbitMQ). ³⁸

Critério	Kafka	RabbitMQ
Modelo	Log de eventos/streams (retenção e replay) ³⁹	Filas/roteamento AMQP ⁴⁰
Melhor para	Analytics/eventos, reprocessamento, alto throughput	Jobs, roteamento, workloads de filas “clássicas”

Critério	Kafka	RabbitMQ
Complexidade	Mais alta (ops, particionamento, tuning)	Menor (principalmente em menor escala)
Observação	Existem garantias e mecanismos como idempotência/transações no ecossistema ⁴¹	Excelente para roteamento e padrões AMQP ⁴²

Para 100k MAU: RabbitMQ (ou até um broker managed simples) pode ser suficiente; Kafka costuma brilhar quando replay/streaming e analytics crescem. ⁴³

APIs, contratos de eventos e guias de implementação

REST vs GraphQL: proposta híbrida (e por quê)

GraphQL é frequentemente adotado quando o cliente precisa navegar relações e reduzir múltiplas chamadas; isso é útil para telas ricas como “página de música com reviews, amigos que ouviram, ranking do mês, comentários”. ⁴⁴

REST continua excelente para comandos e recursos bem definidos, e há guias maduros de design para APIs em larga escala (ex.: guias de design do Google para APIs). ⁴⁵

Recomendação 10/10 - REST para **commands** (write paths): reviews, follow/unfollow, chat send, connect Spotify. - GraphQL para **read paths** complexos: home feed, página de música/álbum, perfil com agregados.

Exemplos de endpoints (REST)

```

POST /auth/register
POST /auth/login
POST /auth/refresh

POST /spotify/connect
POST /spotify/callback
POST /spotify/sync/trigger ; admin/debug

POST /users/{userId}/follow
DELETE /users/{userId}/follow

POST /tracks/{trackId}/reviews
POST /albums/{albumId}/reviews
POST /reviews/{reviewId}/like
POST /reviews/{reviewId}/comments

GET /feed?cursor=...
GET /leaderboards/global?period=2026-03
GET /leaderboards/friends?period=2026-03

```

Exemplo de query GraphQL (read model)

```
query TrackPage($spotifyId: String!, $period: String!) {
  track(spotifyId: $spotifyId) {
    name
    album { name }
    artists { name }
    myReview { rating text }
    friendsReviews { user { username } rating text createdAt }
    leaderboard(period: $period) { rank score }
  }
}
```

Contratos de eventos (exemplos)

Abaixo, contratos base (JSON) para eventos que alimentam feed e agregações. Eles devem ser versionados (ex.: `v1`) e publicados após commit via outbox.

```
{
  "event_id": "uuid",
  "event_type": "TrackReviewed.v1",
  "occurred_at": "2026-03-14T12:34:56Z",
  "actor_user_id": "uuid",
  "data": {
    "track_id": "uuid",
    "track_spotify_id": "4iV5W9uYEdYUVa79Axb7Rh",
    "rating": 4.5,
    "review_id": "uuid",
    "text_len": 128
  }
}
```

```
{
  "event_id": "uuid",
  "event_type": "SpotifySyncCompleted.v1",
  "occurred_at": "2026-03-14T13:10:00Z",
  "actor_user_id": "uuid",
  "data": {
    "sync_kind": "recently_played",
    "items_ingested": 50,
    "window_start": "2026-03-01T00:00:00Z",
    "window_end": "2026-03-14T13:10:00Z"
  }
}
```

Pseudocódigo de worker de sync Spotify (idempotente e resiliente a rate limit)

Observações essenciais: - Access token expira; usar refresh_token para obter novo token. ²⁶
- Respeitar 429 + Retry-After (rate limit em janela móvel). ⁴⁶

```
function runSpotifySyncJob(job):
    account = db.spotify_accounts.get(job.user_id)

    token = tokenService.getValidAccessToken(account.refresh_token)
    # tokenService refreshes when expired (per Spotify docs)

    try:
        if job.kind == "recently_played":
            items = spotifyApi.getRecentlyPlayed(token, limit=50)
        else if job.kind == "top_items":
            items = spotifyApi.getTopTracks(token, time_range="short_term",
            limit=50)

        db.transaction:
            for item in items:
                upsertArtistAlbumTrack(item) # writes to MusicCatalog tables with
                spotify_id unique
                upsertListeningSnapshot(job.user_id, item)
                markJobSuccess(job.id)

            outbox.insert({
                type: "SpotifySyncCompleted.v1",
                actor_user_id: job.user_id,
                data: { kind: job.kind, items_ingested: count(items) }
            })

    catch HttpError as e:
        if e.status == 429:
            waitSeconds = e.headers["Retry-After"] ?? backoff.next()
            reschedule(job, now + waitSeconds)
        else:
            retryOrFail(job, e)
```

Pseudocódigo de agregador de ranking (incremental)

Objetivo: evitar “AVG em tempo real” sobre bilhões de linhas. Em vez disso, materializar `rating_sum` e `rating_count` por período, e atualizar ranking com score derivado (ex.: média, média ponderada, etc.).

```
function onEventTrackReviewed(event):
    month = yyyy_mm(event.occurred_at)

    db.transaction:
        row = db.agg_track_rating_month.get(month, event.data.track_id)
```

```
if row not exists:
    row = { month, track_id, rating_sum=0, rating_count=0 }

row.rating_sum += event.data.rating
row.rating_count += 1
row.avg_rating = row.rating_sum / row.rating_count

db.agg_track_rating_month.upsert(row)

# optional: update Redis sorted set for fast top-N
redis.zadd("lb:tracks:" + month, score=row.avg_rating,
member=event.data.track_id)
```

Operação, segurança, SLOs, custos, benchmarking e roadmap

Segurança: OAuth, JWT e rate limiting

OAuth / PKCE - OAuth 2.0 é o padrão; PKCE mitiga interceptação do authorization code em clientes públicos. ⁴⁷

- O Spotify recomenda Authorization Code Flow para apps de longa duração e PKCE quando client_secret não é seguro em dispositivo. ⁴⁸

JWT - JWT é um padrão (RFC) para transmitir claims; deve ser validado (assinatura, expiração, issuer/audience) e rotacionado quando necessário. ⁴⁹

Rate limiting e consumo irrestrito - OWASP destaca risco de consumo irrestrito (rate limiting / throttling) como controle essencial para disponibilidade. ⁵⁰

Compliance Spotify - Atribuição e link back são exigências explícitas quando exibir conteúdo; e há restrições de uso do conteúdo (inclusive limitações que afetam produtos que tentem virar "provider de metadados" isolado). ²⁴

SLOs e métricas recomendadas (100k MAU)

Definições: - **SLO**: objetivo-alvo medido por SLIs; SLO define error budget. ⁵¹

Sugestão prática de SLOs iniciais (adaptar ao produto): - **API (read)**: p95 < 300ms (rotas cacheadas/feed), p99 < 800ms (páginas ricas). - **API (write)**: p95 < 500ms para postar review. - **Disponibilidade**: 99,9% mensal para endpoints críticos (auth/feed). - **Sync Spotify**: 95% dos jobs concluem em até X minutos (ex.: 15 min) após enfileirados; backlog monitorado. - **Chat**: entrega p95 < 250ms em tempo real (quando ambos online).

Esses SLOs devem gerar **política de error budget**: se o budget for consumido, reduzir deploys arriscados e priorizar confiabilidade. ⁵²

Infra e tecnologias

Base recomendada - Postgres como fonte de verdade (OLTP). ⁵³

- Redis para cache, rate limiting, leaderboards quentes e presença; mas não usar Redis Pub/Sub como única garantia de entrega (semântica at-most-once). ³⁴

- Para busca: Elasticsearch (full-text e discovery). ⁵⁴

- Para analytics: ClickHouse (OLAP colunar) quando necessário. ³⁵

- Containerização com Docker (imagens e portabilidade) e orquestração com Kubernetes/HPA quando o sistema exigir escala automática; Kubernetes descreve automação de deploy/escala/gestão e HPA para aumentar/diminuir pods com carga. ⁵⁵

Chat - WebSocket é protocolo padronizado para comunicação bidirecional. ⁵⁶

- Em .NET, SignalR escala com Redis backplane (documentação oficial). ⁵⁷

Custos estimados (ordem de grandeza) para 100k MAU

Custos variam fortemente por região, tráfego, retenção de eventos e escolha de managed services. As páginas oficiais de pricing e calculadoras são a forma correta de fechar número final. ⁵⁸

Estimativa por cenário (mensal) - MVP enxuto (sem Kafka/ClickHouse/Elastic, com Postgres + Redis + 2-4 nós de app): tipicamente “centenas a poucos milhares” USD/mês ($\approx$ **US\$ 500–3.000/mês**), dependendo de HA/backups e tamanho do Redis.

- **MVP robusto (com mensageria managed + busca + analytics):** normalmente “alguns milhares a dezenas” USD/mês ($\approx$ **US\$ 3.000–15.000/mês**), sobretudo se usar Kafka managed e um cluster Kubernetes pago.

Evidências de custo unitário em serviços comuns: - GKE cobra **taxa fixa por cluster/hora** (ex.: US\$0.10/h) além de compute. ⁵⁹

- AWS EKS tem precificação por “scaling tier” do control plane (valores por hora publicados). ⁶⁰

- Amazon MSK publica exemplos de custo mensal com 3 brokers + storage e mostra estrutura de cobrança (horas de broker + GB-mês + ingest). ⁶¹

- Memorystore (Redis no GCP) publica exemplo de custo por GiB-hora e aproximação mensal. ⁶²

Recomendação pragmática para custo/valor - Iniciar com **containers simples + autoscaling básico**; Kubernetes pode esperar até haver necessidade real (ou usar GKE/GKE Autopilot e pagar a taxa quando justificar). ⁶³

- Evitar Kafka/MSK no dia 1 se RabbitMQ (ou até outbox + polling) suportar; migrar quando houver demanda de replay/streams/analytics pesado. ⁶⁴

Benchmarking com plataformas (e implicações arquiteturais)

Letterboxd (produto de referência) - A própria proposta do Letterboxd inclui atividade e recomendações conforme você segue pessoas, criando um “Activity stream” personalizado. ⁶⁵

- Existe documentação de API em domínio oficial, o que sugere um core bem estruturado para recursos/entidades. ⁶⁶

Last.fm - API fornece métodos como `user.getRecentTracks`, retornando tracks recentes e sinalização de “nowplaying”. Isso valida a importância do conceito de “escuta recente” como primitivo de produto. ⁶⁷

Discogs - API é RESTful e expõe objetos de banco de dados (Artists, Releases etc.), útil como referência de modelagem de catálogo e integração com entidades musicais. 68

Spotify - Documenta explicitamente Authorization, Scopes, Refresh tokens e Rate Limits (429/Retry-After), o que impõe requisitos técnicos claros para sync e resiliência. 69

- Políticas exigem atribuição e link back ao exibir conteúdo e restringem usos (incluindo nota sobre não usar conteúdo em treino de ML em certos contextos de API), o que influencia feature roadmap de "AI discovery". 70

Risco estratégico (dependência de plataforma) - Há histórico público de mudanças abruptas e restrições de endpoints/políticas em APIs de plataformas grandes, o que reforça a necessidade de espelhamento local mínimo e de manter o "core social" do MusicXD independente. 71

Roadmap técnico e plano de escalabilidade

Fase MVP (0 → 100k MAU) - Modular monolith + Postgres + Redis. - SpotifySync por jobs (scheduler + worker). - Feed fan-out on write (inbox) + cache Redis. - Leaderboards por tabela agregada + Redis Sorted Sets. - Outbox + publicação simples (pode ser polling no início) e migração gradual para broker. 37

Fase crescimento (sinais: alto volume de eventos, necessidade de replay/analytics) - Introduzir broker (Kafka/RabbitMQ) e consumidores dedicados. - Introduzir ClickHouse para queries analíticas e segmentações mais complexas. 72

Fase escala avançada - Extrair módulos em serviços quando houver necessidade de deploy independente (ex.: Chat e Analytics). - Evoluir feed para híbrido com limiar alto fanout e re-ranking. - Introduzir Elasticsearch se busca/discovery virar diferencial central. 73

1 17 Bounded Context

https://martinfowler.com/bliki/BoundedContext.html?utm_source=chatgpt.com

2 23 GET /tracks/{id}

https://developer.spotify.com/documentation/web-api/reference/get-track?utm_source=chatgpt.com

3 10 14 48 Authorization Code Flow

https://developer.spotify.com/documentation/web-api/tutorials/code-flow?utm_source=chatgpt.com

4 37 Pattern: Transactional outbox

https://microservices.io/patterns/data/transactional-outbox.html?utm_source=chatgpt.com

5 The Infrastructure Behind Twitter: Scale - Blog - X

https://blog.x.com/engineering/en_us/topics/infrastructure/2017/the-infrastructure-behind-twitter-scale?utm_source=chatgpt.com

6 56 RFC 6455: The WebSocket Protocol

https://www.rfc-editor.org/rfc/rfc6455.html?utm_source=chatgpt.com

7 19 22 53 PostgreSQL: About

https://www.postgresql.org/about/?utm_source=chatgpt.com

8 25 69 Authorization | Spotify for Developers

https://developer.spotify.com/documentation/web-api/concepts/authorization?utm_source=chatgpt.com

- 9 27 **Get Recently Played Tracks - Web API**
https://developer.spotify.com/documentation/web-api/reference/get-recently-played?utm_source=chatgpt.com
- 11 33 **Redis Streams | Docs**
https://redis.io/docs/latest/develop/data-types/streams/?utm_source=chatgpt.com
- 12 30 46 **Rate Limits**
https://developer.spotify.com/documentation/web-api/concepts/rate-limits?utm_source=chatgpt.com
- 13 51 **Defining slo: service level objective meaning**
https://sre.google/sre-book/service-level-objectives/?utm_source=chatgpt.com
- 15 49 **RFC 7519 - JSON Web Token (JWT)**
https://datatracker.ietf.org/doc/html/rfc7519?utm_source=chatgpt.com
- 16 24 70 **Spotify Developer Policy**
https://developer.spotify.com/policy?utm_source=chatgpt.com
- 18 **Use Tactical DDD to Design Microservices - Azure**
https://learn.microsoft.com/en-us/azure/architecture/microservices/model/tactical-domain-driven-design?utm_source=chatgpt.com
- 20 **Uncle Bob's Clean Architecture - Clean Coder Blog**
https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html?utm_source=chatgpt.com
- 21 **Use Domain Analysis to Model Microservices**
https://learn.microsoft.com/en-us/azure/architecture/microservices/model/domain-analysis?utm_source=chatgpt.com
- 26 29 **Refreshing tokens**
https://developer.spotify.com/documentation/web-api/tutorials/refreshing-tokens?utm_source=chatgpt.com
- 28 **Get User's Top Items**
https://developer.spotify.com/documentation/web-api/reference/get-users-top-artists-and-tracks?utm_source=chatgpt.com
- 31 32 **Event Sourcing**
https://martinfowler.com/eaDev/EventSourcing.html?utm_source=chatgpt.com
- 34 **Redis Pub/sub | Docs**
https://redis.io/docs/latest/develop/pubsub/?utm_source=chatgpt.com
- 35 72 **What is ClickHouse?**
https://clickhouse.com/docs/intro?utm_source=chatgpt.com
- 36 54 73 **Full-text search | Elastic Docs**
https://www.elastic.co/docs/solutions/search/full-text?utm_source=chatgpt.com
- 38 39 **Introduction to Apache Kafka**
https://docs.confluent.io/kafka/introduction.html?utm_source=chatgpt.com
- 40 42 **AMQP 0-9-1 Model Explained**
https://www.rabbitmq.com/tutorials/amqp-concepts?utm_source=chatgpt.com
- 41 **Producer Configs | Apache Kafka**
https://kafka.apache.org/41/configuration/producer-configs/?utm_source=chatgpt.com
- 43 64 **What's the Difference Between Kafka and RabbitMQ?**
https://aws.amazon.com/compare/the-difference-between-rabbitmq-and-kafka/?utm_source=chatgpt.com
- 44 **GraphQL | A query language for your API**
https://graphql.org/?utm_source=chatgpt.com

- 45 **Guia de projeto da API | Cloud API Design Guide**
https://docs.cloud.google.com/apis/design?hl=pt-br&utm_source=chatgpt.com
- 47 **RFC 6749 - The OAuth 2.0 Authorization Framework**
https://datatracker.ietf.org/doc/html/rfc6749?utm_source=chatgpt.com
- 50 **API4:2023 Unrestricted Resource Consumption**
https://owasp.org/API-Security/editions/2023/en/0xa4-unrestricted-resource-consumption/?utm_source=chatgpt.com
- 52 **Chapter 2 - Implementing SLOs**
https://sre.google/workbook/implementing-slos/?utm_source=chatgpt.com
- 55 **What is Docker?**
https://docs.docker.com/get-started/docker-overview/?utm_source=chatgpt.com
- 57 **Redis backplane for ASP.NET Core SignalR scale-out**
https://learn.microsoft.com/en-us/aspnet/core/signalr/redis-backplane?view=aspnetcore-10.0&utm_source=chatgpt.com
- 58 **AWS Pricing Calculator**
https://calculator.aws/?utm_source=chatgpt.com
- 59 63 **Google Kubernetes Engine pricing**
https://cloud.google.com/kubernetes-engine/pricing?utm_source=chatgpt.com
- 60 **Amazon EKS Pricing**
https://aws.amazon.com/eks/pricing/?utm_source=chatgpt.com
- 61 **Amazon MSK pricing - Managed Apache Kafka**
https://aws.amazon.com/msk/pricing/?utm_source=chatgpt.com
- 62 **Memorystore for Redis pricing**
https://cloud.google.com/memorystore/docs/redis/pricing?utm_source=chatgpt.com
- 65 **Welcome to Letterboxd • Letterboxd**
https://letterboxd.com/welcome/?utm_source=chatgpt.com
- 66 **Letterboxd API**
https://api-docs.letterboxd.com/?utm_source=chatgpt.com
- 67 **user.getRecentTracks**
https://www.last.fm/api/show/user.getRecentTracks?utm_source=chatgpt.com
- 68 **Home - Discogs API Documentation**
https://www.discogs.com/developers?srsltid=AfmBOorTzVTDcOY0KhcT4VNvj4IsRM7_d_VgC9zMceEKHumTopbXQalb&utm_source=chatgpt.com
- 71 **Spotify suddenly cut off app developers from a bunch of its data**
https://www.theverge.com/2024/12/5/24311523/spotify-locked-down-apis-developers?utm_source=chatgpt.com